



Hooks

Owl hooks [↗](#) are a way to factorize code, even if it depends on some component lifecycle. Most hooks provided by Owl are related to the lifecycle of a component, but some of them (such as **useComponent** [↗](#)) provide a way to build specific hooks.

Using these hooks, it is possible to build many customized hooks that help solve a specific problem, or make some common tasks easier. The rest of this page documents the list of hooks provided by the Odoo web framework.

Name	Short Description
useAssets	load assets
useAutofocus	focus automatically an element referenced by autofocus
useBus	subscribe and unsubscribe to a bus
<u>usePager</u>	Display the pager of the control panel of a view.
usePosition	position an element relative to a target
useSpellCheck	activate spellcheck on focus for input or textarea

useAssets

Location

@web/core/assets

Description

See the section on **lazy loading assets** for more details.

useAutofocus

Location

@web/core/utils/hooks

```
import { useAutofocus } from "@web/core/utils/hooks";

class Comp {
  setup() {
    this.inputRef = useAutofocus();
  }
  static template = "Comp";
}
```

```
<t t-name="Comp">
  <input t-ref="autofocus" type="text"/>
</t>
```

API

useAutofocus()

Returns:

the element reference.

useBus

Location

@web/core/utils/hooks

Description

Add and clear an event listener to a bus. This hook ensures that the listener is properly cleared when the component is unmounted.

```
import { useBus } from "@web/core/utils/hooks";

class MyComponent {
  setup() {
    useBus(this.env.bus, "some-event", event => {
      console.log(event);
    });
  }
}
```

API

`useBus(bus, eventName, callback)`

Arguments:

bus (`EventBus()`) – the target event bus

eventName (`string()`) – the name of the event that we want to listen to

callback (`function()`) – listener callback

usePager

Location

@web/search/pager_hook

Description

Display the **Pager** of the control panel of a view. This hooks correctly sets `env.config` to provide the props to the pager.

```
import { usePager } from "@web/search/pager_hook";

class CustomView {
  setup() {
    const state = owl.hooks.useState({
      offset: 0,
      limit: 80,
      total: 50,
    });
    usePager(() => {
      return {
        offset: this.state.offset,
        limit: this.state.limit,
        total: this.state.total,
        onUpdate: (newState) => {
          Object.assign(this.state, newState);
        },
      };
    });
  }
}
```

API

usePager(*getPagerProps*)

Arguments:

getPagerProps (*function()*) – function that returns the pager props.

usePosition

Location

@web/core/position_hook

Description

Helps positioning an HTML element (the `popper`) relatively to another HTML element (the `reference`). This hook ensures the positioning is updated when the window is resized/scrolls.

```
import { usePosition } from "@web/core/position_hook";
import { Component, xml } from "@odoo/owl";

class MyPopover extends Component {
  static template = xml`
    <div t-ref="popper">
      I am positioned through a wonderful hook!
    </div>
  `;

  setup() {
    // Here, the reference is the target props, which is an HTML element
    usePosition(this.props.target);
  }
}
```

Important

You should indicate your `popper` element using a [t-ref directive](#).

Arguments:

reference (`HTMLElement` or `()=>HTMLElement()`) – the target `HTMLElement` to be positioned from

options (`Options()`) – the positioning options (see table below)

Option	Type	Description
popper	string	this is a useRef reference for the element that will get positioned. Default is "popper" .
container	HTMLElement	the container from which the popper is expected not to overflow. If overflowing occurs, other popper positions are tried until a not overflowing one is found. (default: the <code><html/></code> node)
margin	number	added margin between popper and reference elements (default: 0)
position	Direction[-Variant]	the desired position. It is a string composed of one <code>Direction</code> and one <code>Variant</code> separated by a dash character. <code>Direction</code> could be: <code>top</code> , <code>bottom</code> , <code>right</code> , <code>left</code> . <code>Variant</code> could be: <code>start</code> , <code>middle</code> , <code>end</code> , <code>fit</code> . The variant can be omitted (default variant is <code>middle</code>). The <code>fit</code> variant means that the popper would have the exact same width or height, depending on the chosen direction. Examples of valid positions: <code>right-end</code> , <code>top-start</code> , <code>left-middle</code> , <code>left</code> , <code>bottom-fit</code> . (default position: <code>bottom</code>)

position: PositioningSolution)
 ⇒ void

positioning occurs (e.g. on component mounted/patched, document scroll, window resize...). Can be used i.e. for dynamic styling regarding the current position. The PositioningSolution is an object having the following type:

```
{ direction: Direction, variant: Variant,
  top: number, left: number }
```

Example

```
import { Component, xml, useRef } from "@odoo/owl";
import { usePosition } from "@web/core/position_hook";

class DropMenu extends Component {
  static template = xml`
    <button t-ref="toggler">Toggle Menu</button>
    <div t-ref="menu">
      <t t-slot="default">
        This is the menu default content.
      </t>
    </div>
  `;

  setup() {
    const toggler = useRef("toggler");
    usePosition(
      () => toggler.el,
      {
        popper: "menu",
        position: "right-start",
        onPositioned: (el, { direction, variant }) => {
          el.classList.add(`dm-${direction}`); // -> "dm-top" "dm-right" "dm-bottom"
          el.style.backgroundColor = variant === "middle" ? "red" : "blue";
        },
      },
    );
  }
}
```

useSpellCheck

Location

@web/core/utils/hooks

Description

Activate the spellcheck state to an input or textarea on focus by a `t-ref="spellcheck"` in the current component. This state is then removed on blur, as well as the red outline, which improves readability of the content.

The hook can also be used on any HTML element with the `contenteditable` attribute. To disable spellcheck completely on elements that might be enabled by the hook, set explicitly the `spellcheck` attribute as `false` on the element.

Example

In the following example, the spellcheck will be enabled on the first input, the textarea and the div with `contenteditable="true"`.

```
import { useSpellCheck } from "@web/core/utils/hooks";

class Comp {
  setup() {
    this.simpleRef = useSpellCheck();
    this.customRef = useSpellCheck({ refName: "custom" });
    this.nodeRef = useSpellCheck({ refName: "container" });
  }
  static template = "Comp";
}
```

```
<t t-name="Comp">
  <input t-ref="spellcheck" type="text"/>
  <textarea t-ref="custom"/>
  <div t-ref="container">
    <input type="text" spellcheck="false"/>
    <div contenteditable="true"/>
```

API

useSpellCheck ([options])

Arguments:
options (**Options**()) – the spellcheck options (see table below)

Option	Type	Description
refName	string	this is a useRef reference for the element that will be spellcheck enabled.

 Get Help

- Contact Support
- Ask the Odoo Community

